Signo: Real-Time Isolated Sign Language Recognition

Landmark-Based Temporal Deep Learning in the Browser

5th Semester B.Tech. Computer Science & Engineering | Section A
Machine Learning Laboratory (CSE 3152) | Manipal Institute of Technology, Manipal

Submitted By: John Soni Thomas (CSE-A, Roll No. 40, Reg. No. 240905338)
Mokshagna Y (CSE-A, Roll No. 42, Reg. No. 240905354)

Abstract

Most hearing people do not understand sign language, which creates a communication barrier in places like classrooms and hospitals. In this lab project, we built Signo, a browser application that recognises isolated American Sign Language (ASL) words from a webcam, with all processing on the user's own device. MediaPipe Holistic converts each video frame into 100 numbers describing the hands and upper body, and a Bidirectional LSTM (BiLSTM) classifies how those numbers change over 30 frames. We trained on the ASL Citizen dataset using its official split, in which no signer appears in more than one set. While building the pipeline, we found and fixed a data leakage bug in the original training code, which had merged the validation and test sets. We compared six BiLSTM runs on validation data only, then scored the selected model once on the test set. On 1,000 signs it reached **61.4% top-1 and 84.4% top-5 accuracy** on 3,983 clips from 11 unseen signers, compared with 0.1% and 0.5% for random guessing. A smaller 50-sign model reached 76.7% and 95.9% on its own test set and serves as a fallback. Exported to TensorFlow.js, the 1,000-sign model matched Python's top prediction on all 1,000 test sequences checked. Live webcam accuracy has not been measured, and we report this openly as a limitation.

1. Introduction

Sign languages are complete languages, and they rely on movement over time rather than on single poses. Interpreters are not always available, so a tool that recognises signs could help in everyday situations. Many earlier computer-vision projects focused on static fingerspelled letters, which do not reflect how most signing works.

Building a useful word-level recogniser has two main challenges. First, public word-level datasets such as WLASL [2] and ASL Citizen [1] have few videos per word, so a model working directly from pixels can easily memorise clothing and backgrounds instead of signs. Second, a score only means something if the model is tested on people it has never seen, because that is what happens when a new person stands in front of the camera. This project addresses both challenges, using body landmarks instead of pixels and a strict separation of signers across the data splits.

Signo performs **isolated sign recognition, not continuous translation**. It recognises one word at a time from a fixed list. It does not follow natural signing or produce grammatical sentences.

2. Project Objectives

- **Objective 1:** Convert sign videos into normalised landmark sequences using MediaPipe Holistic, with feature extraction that behaves identically in Python and in the browser.
- **Objective 2:** Train a temporal deep learning model to classify isolated signs, and evaluate it honestly using top-1 accuracy, top-5 accuracy, per-class precision and recall, and unseen signers.
- **Objective 3:** Run the trained model live in a browser, showing the recognised word, the top three guesses and their confidence, and showing no word when the model is unsure.

3. Dataset Description & Anti-Leakage Data Engineering

We used ASL Citizen [1], a community-sourced dataset from Microsoft Research containing 83,399 videos of 2,731 signs from 52 signers. We downloaded only the videos we needed, using HTTP Range requests, so the 45.9 GB archive never had to be fetched in full. About 13 GB was stored, outside the code repository. The dataset's licence does not allow its videos to be redistributed or substantially reproduced, so no dataset video frames appear in this report.

We selected 1,000 signs before any training, so accuracy could not influence the choice. The list keeps 50 common demonstration signs and adds the most common English words among the remaining signs. Of 22,421 selected clips, 22,372 were usable. The other 49 were rejected because too few frames showed a hand.

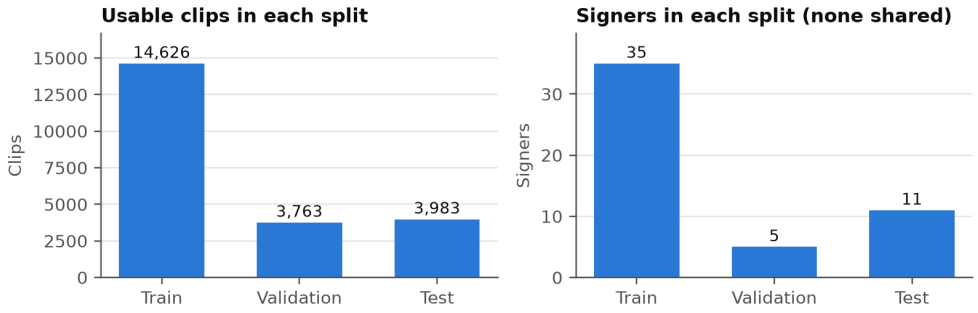


Figure 1: Usable clips and signers in each split of the 1,000-sign dataset.

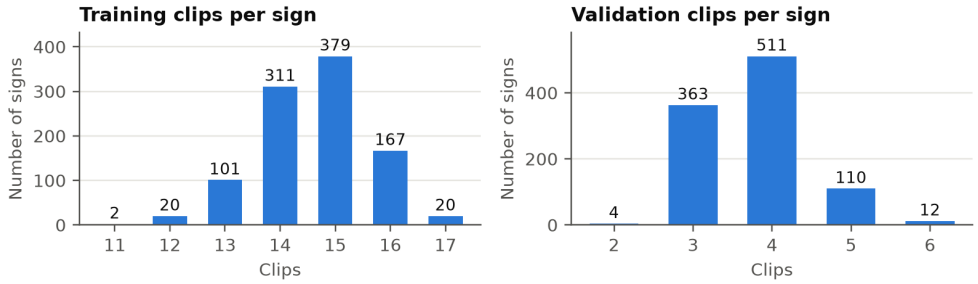


Figure 2: Clips available per sign. Training data is limited, with 11 to 17 clips for each sign.

Preventing Data Leakage

When we inspected the original training code, we found a serious flaw: it merged the validation and test sets into one, then used that set both to decide when to stop training and to report the final score. This is like practising on the final exam and then quoting the mark. We made three changes to fix it:

- **Separate jobs:** `train.py` now uses only training and validation clips. A separate script, `evaluate.py`, scores the test set, logs each attempt and refuses to repeat it by default.
- **Official split:** we kept the dataset's signer assignments, and the training script stops if any signer appears in more than one split. The measured overlap is 0.
- **Automated tests:** a regression test guards against the merged split returning. Other tests check that the Python and browser feature code produce the same output.

One disclosed change: to save about 5 GB of downloads and 1.5 hours of processing, we capped the test split at 4 clips per sign and kept every training and validation clip. The clips were picked using a hash of the file name, not by content or results, and the test signers did not change. Seventeen signs have 3 test clips because an extraction failed.

4. Methodology & Model Architectures

The system has two connected parts, as shown in Figure 3: an offline training pipeline in Python and a live recognition pipeline in the browser. Both use the same feature definition.

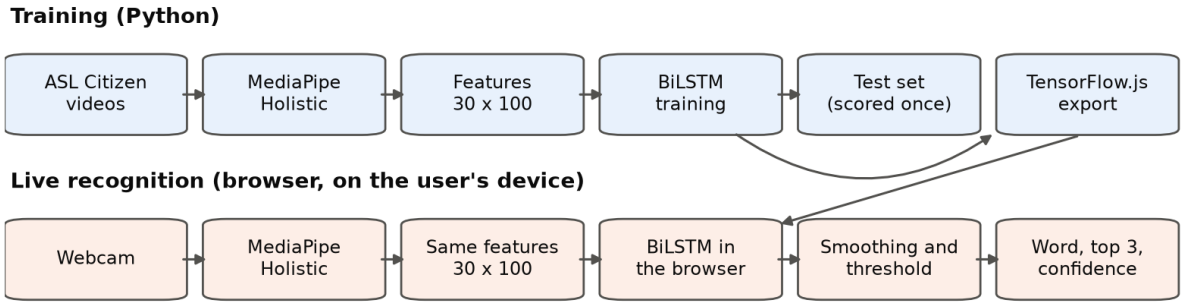


Figure 3: The training pipeline and the live browser pipeline.

Branch A: Landmark Feature Pipeline

MediaPipe [3] Holistic is a pre-trained model that finds points on the body and hands. We use it without training it. Each frame becomes 100 numbers, as listed in Table 1.

Part	Points	Values
Upper body: nose, shoulders, elbows, wrists	7	14
Left hand	21	42
Right hand	21	42
Left and right hand-present flags		2
Total		**100**

Table 1: The 100 features computed for each frame.

Coordinates are centred on the midpoint between the shoulders and divided by shoulder width, so a person's position and distance from the camera matter less. Each clip is trimmed to the part where a hand is visible, keeping 2 extra frames on each side, then resampled to 30 evenly spaced frames. The model's input is therefore a 30 x 100 grid.

Branch B: Bidirectional LSTM Classifier

An LSTM [4] reads a sequence one step at a time and keeps a running memory, which suits signs because their meaning lies in movement. A bidirectional network [5] reads the frames in both directions, so it can use both how a sign begins and how it ends. The selected model is small enough to run live in a browser:

```
Input (30, 100) → Bidirectional LSTM (48) → Dropout (0.3) → Dense (48, ReLU) → Softmax (1,000)
```

It has 110,872 parameters. We trained it with TensorFlow on a laptop CPU in 14.8 minutes, using the Adam optimiser with an initial learning rate of 0.003, batch size 64 and seed 0. The learning rate was halved whenever validation loss stalled for 3 epochs. Each training clip had 4 augmented copies, with small random changes in size, rotation, position and individual points, giving 73,130 training sequences. Augmented copies were made from the landmark numbers and were used for training only.

We trained in two rounds. Round 1 tried 48, 128 and 256 units and kept the epoch with the lowest validation loss. Its logs showed validation accuracy still rising after validation loss had turned upwards, so that rule was keeping weak epochs. Round 2 therefore kept the epoch with the highest validation accuracy, added the falling learning rate, and used stronger dropout (0.5) on the two larger models.

5. Experimental Results & Performance Comparison

All six runs were compared on the same 3,763 validation clips. The test set played no part in this choice.

Run	Units	Dropout	Epoch kept	Val Top-1	Val Top-5	Val Loss	Train Acc.	Parameters
Round 2 (Selected)	**48**	**0.3**	**29 of 39**	**69.5%**	**90.2%**	**1.618**	**92.3%**	**110,872**
Round 2	128	0.5	27 of 31	68.7%	89.3%	1.968	98.4%	396,392
Round 2	256	0.5	12 of 12	66.9%	88.1%	2.251	99.0%	1,119,464
Round 1	48	0.3	14 of 22	66.0%	89.9%	1.498	79.4%	110,872
Round 1	256	0.4	4 of 8	59.6%	85.2%	1.929	85.7%	1,119,464
Round 1	128	0.3	3 of 11	56.3%	83.9%	1.808	69.7%	396,392

Table 2: Validation comparison of all six runs (seed 0).

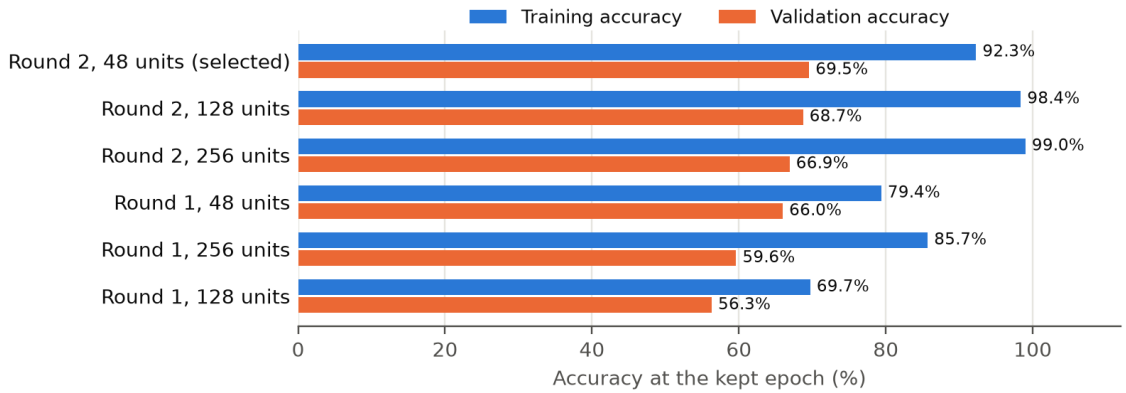


Figure 4: Training and validation accuracy at each run's kept epoch. A wider gap indicates more overfitting.

We selected the 48-unit model from Round 2, which has the highest validation top-1 and top-5 accuracy and the fewest parameters. We then evaluated it once on the test set. Table 3 gives the final results, and they were not tuned afterwards.

Model	Val Top-1	Val Top-5	Test Top-1	Test Top-5	Mean Precision	Mean Recall	Test Clips	Chance Top-1
BiLSTM, 1,000 signs (Proposed)	**69.5%**	**90.2%**	**61.4%**	**84.4%**	**65.0%**	**61.5%**	**3,983**	**0.1%**
BiLSTM, 50 signs (Fallback)	82.3%	96.2%	76.7%	95.9%	79.3%	76.7%	614	2.0%

Table 3: Final performance. The two models use different test sets, so their scores are not directly comparable.

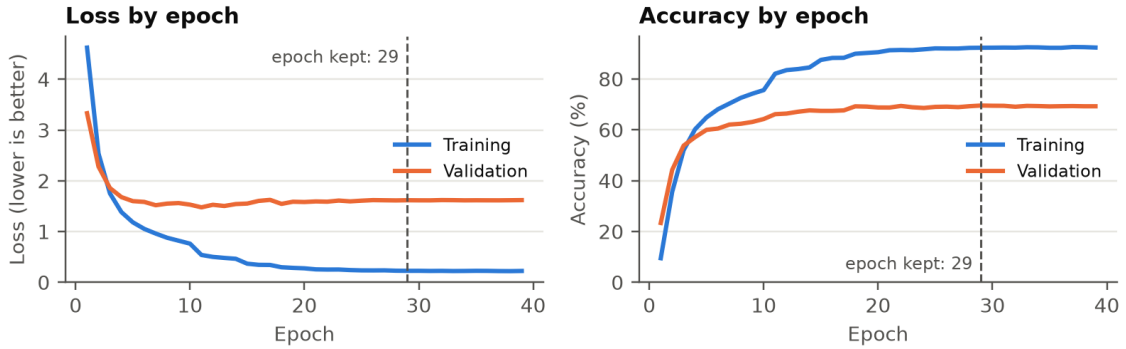


Figure 5: Loss and accuracy of the selected model by epoch. The dashed line marks the kept epoch.

Key findings from the experiments:

1. **Far above chance.** The model's 61.4% test top-1 accuracy compares with 0.1% for random guessing, on signers it never saw. At 84.4%, top-5 accuracy shows that the right sign is often among the leading guesses even when the first guess is wrong.

2. **Larger models did not help within the time available.** In Round 2, more units brought higher training accuracy (up to 99.0%) but lower validation accuracy. However, the 128-unit and 256-unit runs hit a 25-minute limit, and the 256-unit run's best epoch was its last. This result does not show that a larger model could never do better.

3. **The selected model overfits.** Its training accuracy of 92.3% compares with 69.5% on validation. With only 11 to 17 real clips per sign, this is not surprising.

4. **Test accuracy is 8.1 points below validation.** We chose the best of six runs using validation data, which makes that score somewhat optimistic. The validation set also contains only 5 signers, and the test set contains 11 different ones. The test score is the figure to rely on.

5. **Per-sign scores vary widely.** As Figure 6 shows, 198 signs reached 100% recall and 45 reached 0%. Each test clip usually moves a sign's recall by 25%, so single-sign scores are rough. Among common words, *family* and *home* scored 100% precision and recall, and *stop*, *thankyou* and *think* scored 0%.

6. **Many errors look reasonable.** The most frequent confusions include *favorite* read as *color*, *haha* as *three*, *think* as *mind*, *stars* as *star*, *sit* as *chair* and *equal* as *even*. Several of these pairs are related in meaning. No single confusion accounts for more than 4 clips.

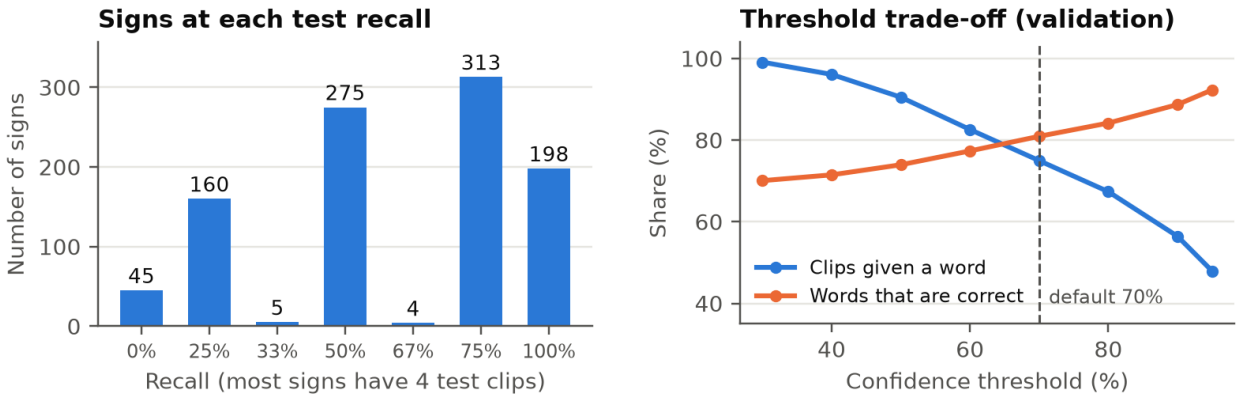


Figure 6: Left, the number of signs at each test recall. Right, the effect of the confidence threshold on validation clips.

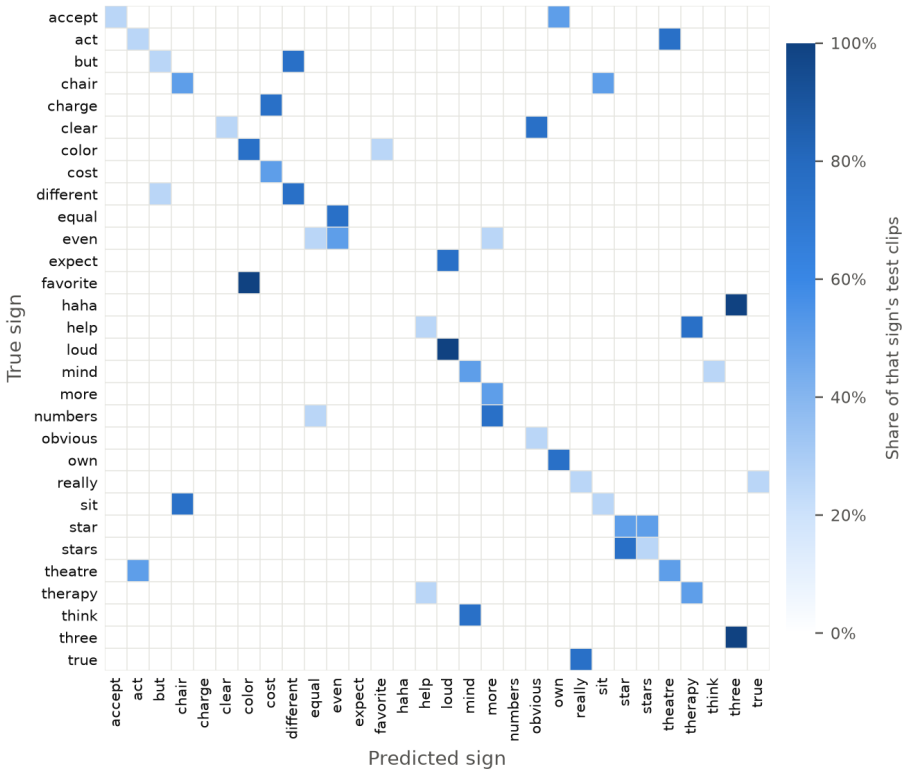


Figure 7: Test confusion matrix for 30 signs taken from the most frequent confusions. Errors involving other signs are not shown. The full 1,000 x 1,000 matrix is in `results_1000/confusion_matrix.csv`.

6. Browser Integration & Real-Time Verification

We exported the model to TensorFlow.js. The app loads it automatically on startup and checks its input shape, output size and a trial prediction before using it. About 5 times per second, it takes the last 2.2 seconds of frames, trims them, resamples them to 30 frames and runs the model. Each prediction took about 67 ms on our laptop in a test browser session.

To prevent flickering and overconfident answers, the last 3 predictions are averaged. A word appears only if its confidence passes the threshold (70% by default) and it stays on top for 2 consecutive checks. Otherwise the app shows "Not sure yet", and with no hands visible it shows "No hands in view". On validation clips, the 70% threshold gives a word for 75% of clips, and 81% of those words are correct. At 90%, these figures become 56% and 89%, as shown in

Figure 6.

Check	Result	Status
TF.js vs Python on identical test sequences	Same top-1 and top-3 order on 1,000 of 1,000. Largest probability difference under 0.000002	Passed
Test videos through browser MediaPipe, features and model	60 clips: 65% top-1 and 85% top-5. Python scored 65% top-1 on the same clips. Same prediction on 53 of 60	Passed
Model switching and fallback	Labels and output size stay matched. If the 1,000-sign model is missing or corrupt, the app opens with the 50-sign model	Passed
Live app loop with test videos as a simulated camera	Input size correct (3,000 values). Over three runs of 8 clips: 5 or 6 right, 0 or 1 wrong, and the rest gave no word. Six signs were picked for good test results, so this is not an accuracy score	Passed as a functional check
Real webcam, 50-sign model	Informal trial: some words were recognised correctly. No counts were recorded	Not measured
Real webcam, 1,000-sign model	No measured trial	Not measured

Table 4: Verification of the browser application.

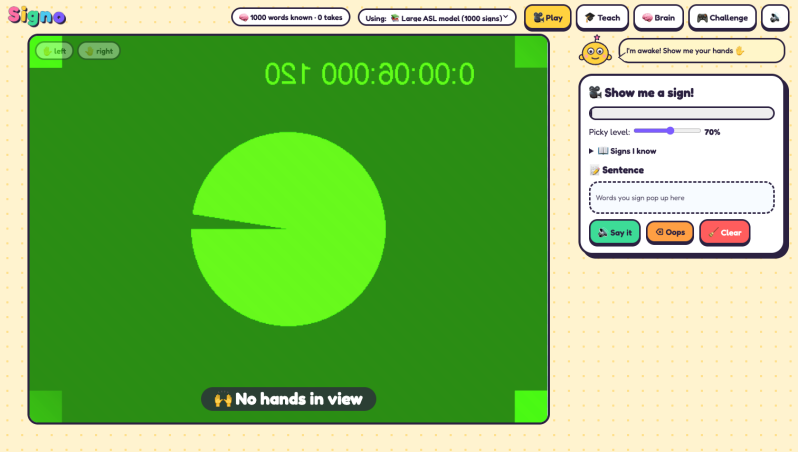


Figure 8: The Signo interface. The camera feed here is Chrome's built-in test pattern, so the app correctly reports that no hands are in view.

7. Practical Deployment: Two-Model Browser Architecture

Signo is a static web page, so it needs no application server. Video stays on the user's device, which matters because sign videos show people's faces. An internet connection is needed only to load TensorFlow.js and MediaPipe from their content delivery network. The app holds two models side by side:

- **Default (1,000 signs):** the larger vocabulary, with lower accuracy because there are more signs to confuse.
- **Fallback (50 signs):** a smaller vocabulary with higher test accuracy, selectable from a menu and loaded automatically if the larger model fails.

Known Limitations

- **Live accuracy is unknown:** all test scores come from fluent signers in dataset videos. A beginner's signing at a webcam may score lower.
- **No facial information:** the nose is the only face point used, although facial expressions carry real meaning in ASL.
- **Limited evidence:** each final model comes from a single training run and seed, so we report no mean or standard deviation.
- **No "not signing" class:** the app relies on hand visibility and the threshold, so any hand movement is treated as an attempted sign.
- **Fixed, unfiltered vocabulary:** signs outside the list are forced towards the closest known sign. The list was chosen by word frequency and includes some offensive words.

8. Conclusion & Future Work

In this project, we built a working pipeline from raw sign videos to word-level recognition in a browser. We found and fixed a validation and test leakage bug, kept signers separate across splits, chose the model using validation data only and scored the test set once. The selected BiLSTM reached 61.4% top-1 and 84.4% top-5 accuracy on 1,000 signs from unseen signers, and the browser version matched Python's top prediction on all 1,000 sequences checked. The main open question is how well it works on a real webcam, which we have not yet measured.

For future work, we plan to measure live webcam accuracy with real users, add more training clips per sign, repeat training with several seeds, give larger models longer training runs, add face landmarks and a "not signing" class, and eventually move towards continuous recognition.

References

- **[1]** A. Desai et al., 'ASL Citizen: A Community-Sourced Dataset for Advancing Isolated Sign Language Recognition', Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, 2023.
- **[2]** D. Li, C. Rodriguez Opazo, X. Yu, and H. Li, 'Word-level Deep Sign Language Recognition from Video: A New Large-scale Dataset and Methods Comparison', IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2020.
- **[3]** C. Lugaresi et al., 'MediaPipe: A Framework for Building Perception Pipelines', arXiv:1906.08172, 2019.
- **[4]** S. Hochreiter and J. Schmidhuber, 'Long Short-Term Memory', Neural Computation, vol. 9, no. 8, 1997.
- **[5]** M. Schuster and K. K. Paliwal, 'Bidirectional Recurrent Neural Networks', IEEE Transactions on Signal Processing, vol. 45, no. 11, 1997.